

GitHub Rate Limit Fix for Coolify Deployments

Problem

Composer downloads Laravel dependencies from GitHub during Docker builds. GitHub enforces rate limits:

- **Anonymous requests:** 60 per hour per IP
- **Authenticated requests:** 5,000 per hour

When Coolify's build server exceeds the anonymous limit, deployments fail with:

```
Failed to download ... from dist: ... (HTTP/2 429)
ERROR: process "/bin/sh -c composer install..." did not complete successfully: exit
code: 100
```

Solution

Configure Composer to authenticate with GitHub using a Personal Access Token (PAT).

Step 1: Create a GitHub Personal Access Token

1. Go to GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)**
 - Direct link: <https://github.com/settings/tokens>
 2. Click **Generate new token** → **Generate new token (classic)**
 3. Set:
 - **Note:** Coolify Composer Builds
 - **Expiration:** 90 days (or No expiration for long-term use)
 - **Scopes:** **NO scopes needed** (leave all checkboxes unchecked)
 - Composer only needs read access to public repos, which doesn't require any scopes
 4. Click **Generate token**
 5. **Copy the token** (starts with `ghp_...`) — you won't see it again
-

Step 2: Add Token to Coolify as Build Argument

For the API (`marqira-api`)

1. Open Coolify → **Projects** → **MarQira Pulse** → **marqira-api**
2. Go to the **Environment Variables** or **Build** section
3. Add a **Build Argument**:
 - **Name:** `GITHUB_TOKEN`

- **Value:** `ghp_...` (paste your token)
- **Type:** Build Argument (or Build-time secret if available)

4. Save

For the Dashboard (`marqira-dashboard`)

If the dashboard Dockerfile also uses Composer/npm with GitHub dependencies:

1. Follow the same steps as above for the `marqira-dashboard` service
2. Add the same `GITHUB_TOKEN` build argument

Step 3: Redeploy

1. In Coolify, click **Deploy** (or **Redeploy**) for the `marqira-api` service
2. The build will now authenticate with GitHub and bypass rate limits
3. Repeat for `marqira-dashboard` if needed

Verification

Check the build logs in Coolify. You should see:

```
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist
```

Successfully downloading all packages without any `HTTP/2 429` errors.

Alternative: Wait and Retry

If you don't want to set up a token immediately:

- GitHub rate limits reset **every hour**
- Simply wait 30-60 minutes and retry the deployment
- This is fine for one-off deployments, but the token prevents future issues

Security Notes

- The token has **no permissions** (no scopes selected), so it cannot access private repos or perform any write operations
 - It's only used to increase Composer's download rate limit
 - Coolify stores build arguments securely
 - Set an expiration (90 days recommended) and renew when needed
-

Troubleshooting

“Still getting 429 errors”

- Verify the token is correctly set as a **Build Argument** (not a runtime environment variable)
- Check the token hasn't expired
- Ensure the token was copied completely (starts with `ghp_`)

“Build works without the token locally”

- That's normal — the Dockerfile works with or without the token
 - The token is only needed when the build server's IP has hit the rate limit
 - Local development rarely hits the limit since you're not rebuilding constantly
-

How It Works

The Dockerfile now includes:

```
ARG GITHUB_TOKEN
RUN if [ -n "$GITHUB_TOKEN" ]; then \
    composer config -g github-oauth.github.com "$GITHUB_TOKEN"; \
fi
```

- If `GITHUB_TOKEN` is provided, Composer authenticates with GitHub
- If not provided, Composer falls back to anonymous downloads
- This keeps local development simple while fixing production rate limits